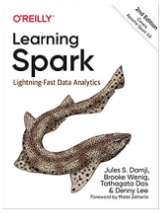
09.2: Apache Spark: Primitives

UMD DATA605 - Big Data Systems

09.2: Apache Spark: Primitives

– **Instructor**: Dr. GP Saggese, gsaggese@umd.edu
– @References@:
  – Key concepts discussed in the slides
  – Academic paper
    – "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing", 2012
  – Mastery
    – "Learning Spark: Lightning-Fast Data Analytics" (2nd Edition)
    – Not my favorite, but free

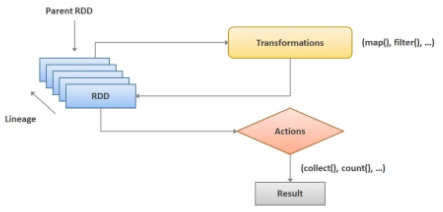


1 / 16

2 / 16: Transformations vs Actions

Transformations vs Actions

- **Transformations**
  - Transform a Spark RDD into a new RDD without modifying input data
  - Immutability like functional programming
  - E.g., `select()`, `filter()`, `join()`, `orderBy()`



- Transformations are **evaluated lazily**
  - Spark optimizes the computation by analyzing the workload
    - E.g., joining, pipeline operations, breaking into stages
  - Record results as "lineage"
    - The sequence of stages is rearranged and optimized without changing the results
- **Actions**
  - Trigger computation evaluation
  - E.g., `show()`, `take()`, `count()`, `collect()`, `save()`

2 / 16

- **Transformations**
  - Transformations in Spark are operations that create a new RDD (Resilient Distributed Dataset) from an existing one without altering the original data. This concept is similar to immutability in functional programming, where data remains unchanged.
  - Examples of transformations include operations like `select()`, `filter()`, `join()`, and `orderBy()`. These operations allow you to manipulate and refine data as needed.
- **Transformations are evaluated lazily**
  - Lazy evaluation means that transformations are not immediately executed. Instead, Spark waits until an action is called to execute the transformations. This allows Spark to optimize the computation by analyzing the entire workload.
  - Spark records the sequence of transformations as "lineage," which helps in optimizing and rearranging the stages of computation without affecting the final result.
- **Actions**
  - Actions are operations that trigger the execution of transformations and return a result to the driver program or write it to storage.
  - Examples of actions include `show()`, `take()`, `count()`, `collect()`, and `save()`. These operations are essential for retrieving or saving the results of your computations.

3 / 16: Spark Example: MapReduce in 1 or 4 Line

Spark Example: MapReduce in 1 or 4 Line

```
!more data.txt
```
executed in 1.77s, finished 04:37:35 2022-11-23

```
One a penny, two a penny, hot cross buns
```

```
lines = sc.textFile("data.txt").flatMap(lambda line: line.split(" "))
pairs = lines.map(lambda s: (s, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
result = counts.collect()
print(result)
```
executed in 428ms, finished 04:36:24 2022-11-23

```
[('One', 1), ('two', 1), ('hot', 1), ('cross', 1), ('a', 2), ('penny,', 2), ('buns', 1)]
```

MapReduce in 4 Spark lines

```
result = sc.textFile("data.txt").flatMap(lambda line: line.split(" ")).map(
    lambda s: (s, 1)).reduceByKey(lambda a, b: a + b).collect()
print(result)
```
executed in 591ms, finished 05:06:00 2022-11-23

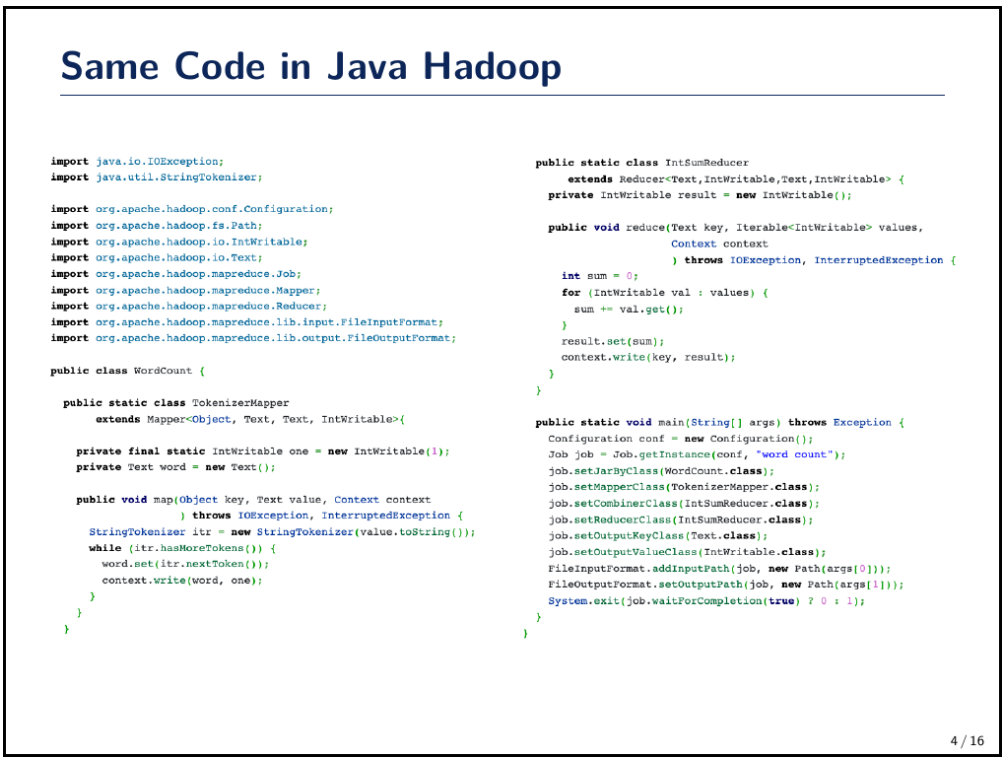
```
[('One', 1), ('two', 1), ('hot', 1), ('cross', 1), ('a', 2), ('penny,', 2), ('buns', 1)]
```

MapReduce in 1 (show-off) line

3 / 16

- **Spark Example: MapReduce in 1 or 4 Lines**
- **Context**: This slide demonstrates how Apache Spark can simplify the MapReduce programming model, which is traditionally used for processing large data sets with a distributed algorithm on a cluster.
- **MapReduce in 4 Spark Lines**:
  - **Data Loading**: The `sc.textFile("data.txt")` command reads the text file into an RDD (Resilient Distributed Dataset).
  - **Splitting Lines**: `flatMap(lambda line: line.split(" "))` splits each line into words.
  - **Mapping**: `map(lambda s: (s, 1))` creates a key-value pair for each word with an initial count of 1.
  - **Reducing**: `reduceByKey(lambda a, b: a + b)` aggregates the counts for each word.
  - **Collecting Results**: `collect()` gathers the results into a list for display.
- **MapReduce in 1 (Show-off) Line**:
  - This version condenses the entire process into a single line, showcasing Spark's ability to perform complex operations concisely.
  - The operations are chained together, demonstrating the power and flexibility of functional programming in Spark.
- **Output**: Both examples produce the same result, a list of tuples with each word and its count, illustrating the efficiency and simplicity of Spark for data processing tasks.

4 / 16: Same Code in Java Hadoop



- **Imports and Setup**
  - The code begins by importing necessary Java and Hadoop libraries. These include classes for handling I/O exceptions, tokenizing strings, and various Hadoop-specific classes for configuration, file paths, and the MapReduce framework.
  - *Key Libraries*: `org.apache.hadoop.mapreduce` for MapReduce operations, `org.apache.hadoop.io` for data types like `IntWritable` and `Text`.
- **WordCount Class**
  - This is the main class that contains the logic for the MapReduce job. It includes two inner classes: `TokenizerMapper` and `IntSumReducer`.
- **TokenizerMapper Class**
  - **Purpose**: This class extends the `Mapper` class and is responsible for the map phase of the MapReduce job.
  - **Functionality**: It tokenizes each line of input text into words. For each word, it emits a key-value pair where the key is the word and the value is the integer `1`.
  - **Key Methods**:
    - `map`: Uses `StringTokenizer` to split the input text into words and writes each word with a count of one to the context.
- **IntSumReducer Class**
  - **Purpose**: This class extends the `Reducer` class and handles the reduce phase.
  - **Functionality**: It sums up the counts for each word received from the mapper.
  - **Key Methods**:
    - `reduce`: Iterates over the values associated with a word and sums them up, then writes the result to the context.
- **Main Method**

- **Purpose**: Sets up and runs the MapReduce job.
- **Configuration**:
  - Creates a new `Configuration` object and a `Job` instance.
  - Sets the job's jar, mapper, combiner, and reducer classes.
  - Specifies the output key and value classes.
- **Input/Output Paths**: Uses `FileInputFormat` and `FileOutputFormat` to set the input and output paths from command-line arguments.
- **Execution**: Calls `job.waitForCompletion(true)` to execute the job and exits based on the job's success.

5 / 16: Spark Example: Logistic Regression in MapReduce

Spark Example: Logistic Regression in MapReduce

- Logistic Regression

```
# Load points
points = spark.textFile(...).map(parsePoint).c

# Initial separating plane
w = numpy.random.ranf(size=D)

# Until convergence
for i in range(ITERATIONS):
    # Parallel loop over the samples i=1...m
    gradient = points.map(
        lambda p:
          (1 / (1 + exp(-p.y*(w.dot(p.x))))) -
          p.y * p.x
    ).reduce(lambda a, b: a + b)
    w -= alpha * gradient

print("Final separating plane: %s" % w)
```

Repeat until convergence {

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

Repeat {

$$\theta_j := \theta_j - \frac{\alpha}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

}

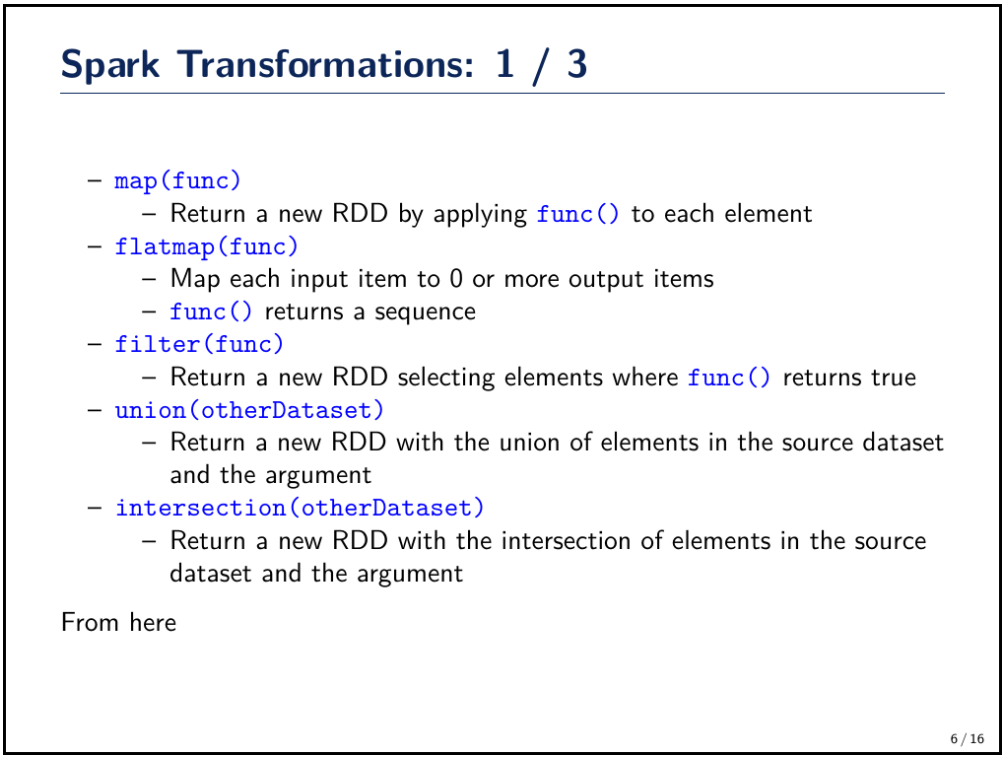
5 / 16

- **Logistic Regression in Spark**
  - This example demonstrates how to implement logistic regression using Spark and MapReduce.
  - **Loading Data**: The data points are loaded using `spark.textFile()` and parsed with `parsePoint`. The `cache()` function is used to store the data in memory for faster access.
  - **Initial Weights**: A random initial separating plane `w` is created using `numpy.random.ranf()`, which generates random numbers.
  - **Iterative Process**: The algorithm iterates until convergence is achieved.
    - **Gradient Calculation**: For each data point, the gradient is calculated using a lambda function. This involves computing the logistic function and adjusting based on the label `p.y`.
    - **Weight Update**: The weights `w` are updated by subtracting the product of the learning rate `alpha` and the gradient.
  - **Output**: The final separating plane is printed, representing the model's decision bound-

ary.

- **Mathematical Context**
  - The equations illustrate the gradient descent process used in logistic regression.
  - **Gradient Descent**: The weights are updated iteratively to minimize the cost function `J( )`.
  - **Cost Function**: `J( )` measures the error between predicted and actual values, guiding the optimization process.
  - **Convergence**: The process repeats until the change in weights is minimal, indicating convergence.

6 / 16: Spark Transformations: 1 / 3

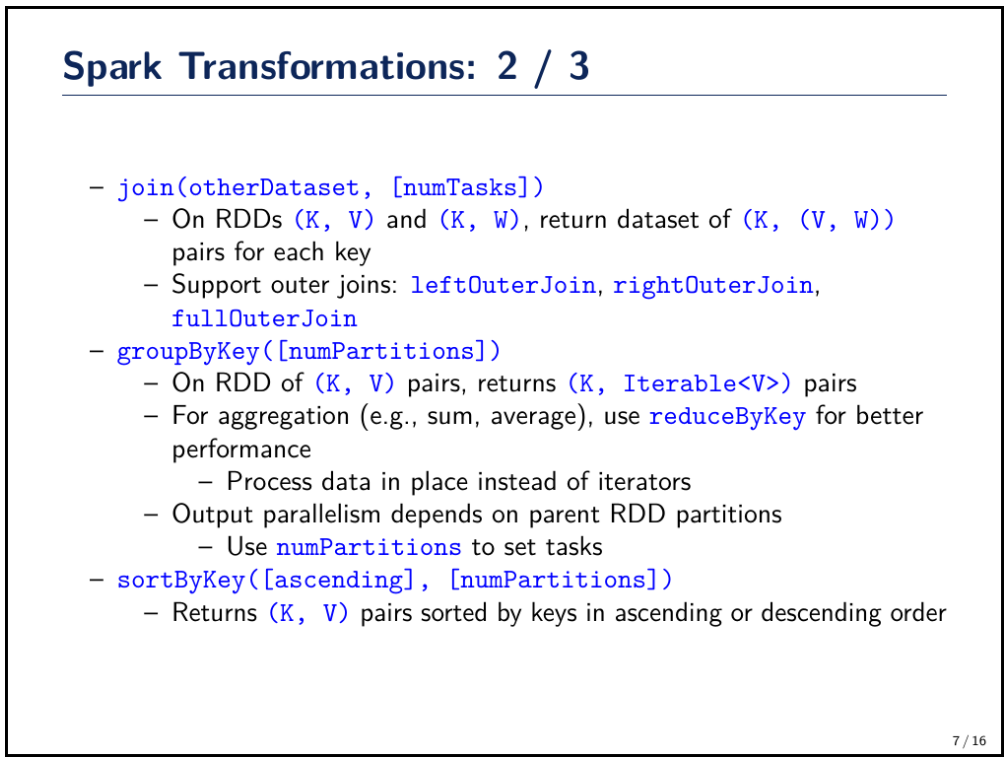


- `map(func)`
  - This transformation is used to apply a function, `func()`, to each element of an RDD (Resilient Distributed Dataset). The result is a new RDD where each element is the result of the function applied to the corresponding element in the original RDD. This is useful for transforming data, such as converting all strings to uppercase or multiplying numbers by a constant.
- `flatmap(func)`
  - Similar to `map`, but with a key difference: `flatmap` allows each input item to be mapped to zero or more output items. The function `func()` returns a sequence, and these sequences are then flattened into a single RDD. This is particularly useful when you want to split strings into words or expand lists into individual elements.
- `filter(func)`
  - This transformation creates a new RDD by selecting only the elements that satisfy a given condition, defined by `func()`. If `func()` returns true for an element, that element

is included in the new RDD. This is helpful for narrowing down data to only the relevant parts, such as filtering out negative numbers or selecting records from a specific year.

- `union(otherDataset)`
  - The `union` transformation combines two RDDs into one, containing all elements from both the source dataset and the specified `otherDataset`. This is useful when you want to merge datasets, such as combining logs from different sources.
- `intersection(otherDataset)`
  - This transformation creates a new RDD containing only the elements that are present in both the source dataset and the `otherDataset`. It is useful for finding common elements between datasets, such as identifying shared customers between two lists.
- **Reference**
  - The transformations discussed are part of Apache Spark's RDD programming guide, which provides detailed information on how to manipulate and process large datasets efficiently using Spark's distributed computing capabilities.

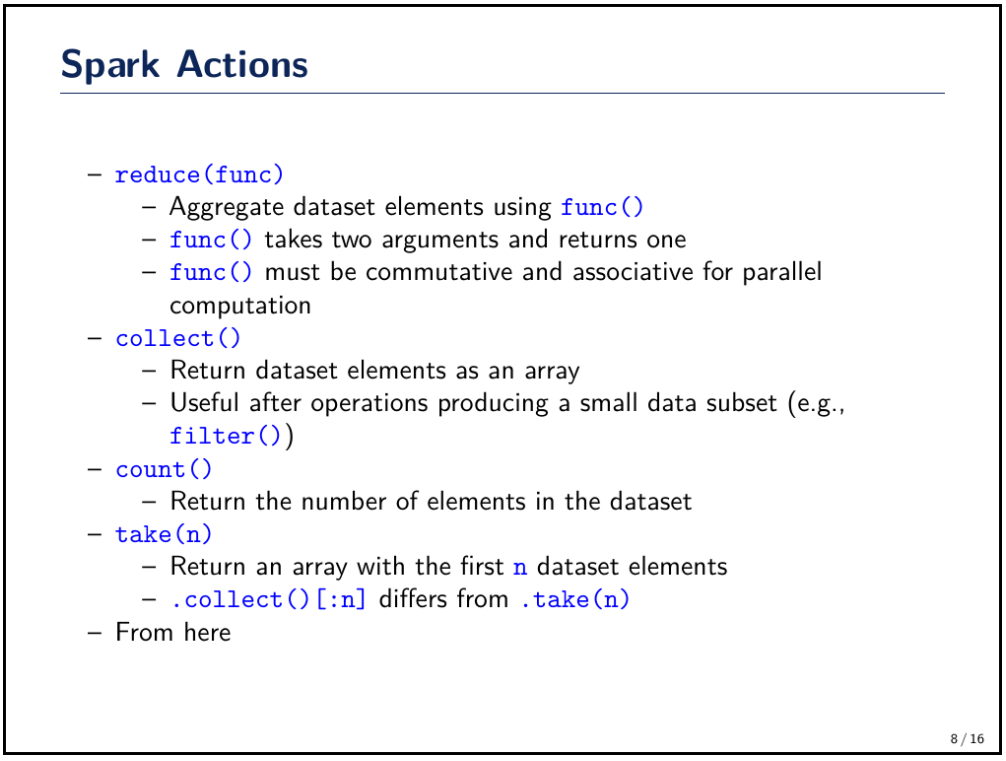
7 / 16: Spark Transformations: 2 / 3



- `join(otherDataset, [numTasks])`
  - This transformation is used when you have two datasets, each containing key-value pairs, and you want to combine them based on their keys. For example, if you have one dataset with user IDs and their names, and another with user IDs and their purchase history, a join will allow you to pair each user with their purchase history.
  - It supports different types of joins, such as *leftOuterJoin*, *rightOuterJoin*, and *fullOuterJoin*. These allow you to include keys that are only present in one of the datasets, depending on the type of join you choose.
- `groupByKey([numPartitions])`

  - This transformation is used to group all values associated with the same key into a single collection. For instance, if you have a dataset of sales transactions, you can group all transactions by the store ID.
  - While `groupByKey` is useful, it can be inefficient for aggregations like sum or average because it shuffles all data with the same key across the network. Instead, `reduceByKey` is recommended for such operations as it combines values locally before shuffling, which is more efficient.
  - The number of partitions in the output can be controlled with `numPartitions`, which affects how the data is distributed across the cluster.
- `sortByKey([ascending], [numPartitions])`
  - This transformation sorts the dataset based on the keys. You can choose to sort in ascending or descending order, which is useful when you need to organize data for reporting or further processing.
  - The sorting operation can be parallelized across multiple partitions, and you can specify the number of partitions to control the level of parallelism and potentially improve performance.

8 / 16: Spark Actions



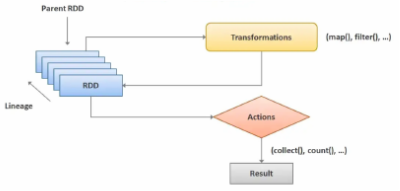
- **Spark Actions**
  - In Apache Spark, *actions* are operations that trigger the execution of the computation graph and return a result to the driver program. They are crucial because they produce the final output of the transformations applied to the data.
- `reduce(func)`
  - This action is used to *aggregate* the elements of a dataset using a specified function, `func()`.

  - The function `func()` should take two arguments and return a single value. This means it combines two elements at a time.
  - It's important that `func()` is both *commutative* (order doesn't matter) and *associative* (grouping doesn't matter) to ensure correct results in parallel computation, which is a key feature of Spark.
- `collect()`
  - This action returns all the elements of the dataset as an array to the driver program.
  - It's particularly useful when you have a small subset of data after transformations like `filter()`, as it allows you to work with the data locally.
- `count()`
  - This action simply returns the total number of elements present in the dataset.
  - It's a straightforward way to get a quick overview of the dataset size.
- `take(n)`
  - This action returns an array containing the first `n` elements of the dataset.
  - It's different from using `.collect()[:n]` because `.take(n)` is optimized to retrieve only the necessary elements, making it more efficient for large datasets.
- **From here**
  - This reference points to the official Spark documentation, which is a valuable resource for understanding the details and best practices of using Spark actions and transformations.

9 / 16: Spark: Fault-tolerance

Spark: Fault-tolerance

- Spark leverages *immutability* and *lineage* for fault tolerance
- In case of **failure**
  - Reproduce RDD by replaying the lineage
  - Checkpoints aren't needed
  - Keep data in memory for increased performance
- Fault-tolerance is free!



9 / 16

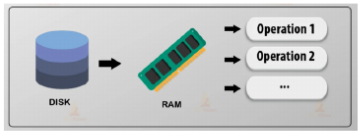
- **Spark leverages *immutability* and *lineage* for fault tolerance**
  - Spark uses Resilient Distributed Datasets (RDDs) which are immutable. This means once an RDD is created, it cannot be changed. This immutability helps in maintaining consistency and reliability.

  - Lineage refers to the sequence of transformations that were applied to the data. By keeping track of these transformations, Spark can recreate any lost data.
- **In case of failure**
  - If a failure occurs, Spark can reconstruct the lost RDD by replaying the lineage of transformations. This means it can rebuild the data from the original source or previous transformations without needing to store intermediate data.
  - Checkpoints, which are snapshots of data, aren't necessary for fault tolerance in Spark because of its ability to use lineage. However, they can still be used for optimization in certain scenarios.
  - Keeping data in memory enhances performance, as accessing data from memory is faster than from disk.
- **Fault-tolerance is free!**
  - The design of Spark inherently provides fault tolerance without additional overhead, making it efficient and robust for large-scale data processing.

10 / 16: Spark: RDD Persistence

Spark: RDD Persistence

- **Users explicitly cache an RDD**
  - I.e., `persist()`, `unpersist()`
  - Cache if RDD is expensive to compute
    - E.g., filtering large data
  - When you persist an RDD, each node:
    - Stores partitions of the RDD (in memory or on disk)
    - Reuses cached partitions on derived datasets



- **Cache**
  - Enhances speed of future actions (often by >10x)
  - Managed by Spark using LRU policy + garbage collector
- **Users can choose the storage level**
  - `MEMORY_ONLY` (default)
  - `DISK_ONLY` (e.g., Python Pickle)
  - `MEMORY_AND_DISK`
    - If an RDD doesn't fit in memory, store it on disk
  - Caching on disk can be more costly than not caching

10 / 16

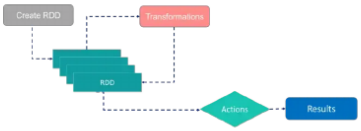
- **Users explicitly cache an RDD**
  - *persist()* and *unpersist()* are methods used to manage RDD storage.
  - Caching is beneficial when an RDD is costly to compute, such as when filtering large datasets.
  - When an RDD is persisted, each node in the cluster stores its partitions either in memory or on disk.
  - Cached partitions can be reused in subsequent computations, improving efficiency.
- **Cache**
  - Caching significantly speeds up future actions, often by more than ten times.

  - Spark manages cached data using a Least Recently Used (LRU) policy combined with garbage collection to free up space.
- **Users can choose the storage level**
  - The default storage level is `MEMORY_ONLY`, which keeps data in RAM for fast access.
  - `DISK_ONLY` stores data on disk, which is slower but useful for large datasets.
  - `MEMORY_AND_DISK` is a hybrid approach, storing data on disk if it doesn't fit in memory.
  - Storing data on disk can be more resource-intensive than not caching at all.
  - It's generally inefficient to cache everything, as it can lead to unnecessary resource usage.

11 / 16: Spark: RDD Persistence and Fault-tolerance

Spark: RDD Persistence and Fault-tolerance

- Spark unifies persistence and fault-tolerance through RDD lineage
- Caching and Persistence
  - Store RDDs in memory or disk to avoid repeated computation
  - Useful for iterative algorithms and interactive queries
  - E.g., `cache()` (memory only) or `persist()` (custom storage level)



- Fault-Tolerance Mechanism
  - RDDs are immutable and record lineage of transformations
  - If a partition is lost, Spark reconstructs it using lineage
  - No need to checkpoint unless lineage is too long
- Persistence is Fault-Tolerant
  - Cached RDDs can be recomputed from lineage if data is lost
  - Ensures recovery without manual intervention

11 / 16

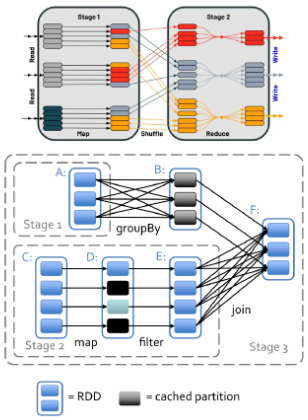
- **Spark unifies persistence and fault-tolerance through RDD lineage**
  - Spark uses a concept called *lineage* to manage both persistence and fault-tolerance. This means that it keeps track of all the transformations applied to an RDD (Resilient Distributed Dataset) so it can recreate it if needed.
- **Caching and Persistence**
  - *Caching* and *persistence* are techniques to store RDDs in memory or on disk. This helps avoid recalculating them, which is especially useful for tasks that require repeated access to the same data, like iterative algorithms or interactive queries.
  - You can use `cache()` to store RDDs in memory or `persist()` to specify a custom storage level, such as memory and disk.
- **Fault-Tolerance Mechanism**
  - RDDs are *immutable*, meaning they cannot be changed once created. Spark records the lineage of transformations, which allows it to reconstruct any lost partitions.
  - Checkpointing is generally unnecessary unless the lineage becomes too long, which could make recovery inefficient.

- **Persistence is Fault-Tolerant**
  - Even if cached RDDs are lost, Spark can recompute them using the lineage information. This ensures that data recovery happens automatically without requiring manual intervention.

12 / 16: Spark Shuffle

Spark Shuffle

- Certain Spark operations trigger a data shuffle
- E.g., **reduceByKey()**
  - Combine values `[v1, ..., vn]` for key `k` into `(k, v)` where `v = reduce(v1, ..., vn)`
  - Values for a key must be on the same partition/machine
- **Data shuffle** = re-distribute data across partitions/machines
- **Data shuffle is expensive** because of:
  - Data serialization (pickle)
  - Disk I/O (saving to disk)
  - Network I/O (copying across Executors)
  - Deserialization and memory allocation
- **Spark schedules general task graphs**
  - Automatic function pipelining
  - Data locality aware



12 / 16

- **Certain Spark operations trigger a data shuffle**
  - Some operations in Spark, like `reduceByKey()`, require data to be shuffled. This means data is moved around to ensure that all values for a specific key are on the same partition or machine. This is crucial for operations that need to combine or aggregate data by key.
- **Data shuffle = re-distribute data across partitions/machines**
  - Shuffling involves redistributing data across different partitions or machines to ensure that operations can be performed efficiently. This is a key part of how Spark handles large-scale data processing.
- **Data shuffle is expensive because of:**
  - *Data serialization*: Converting data into a format that can be easily stored or transmitted.
  - *Disk I/O*: Writing data to disk, which can be slow.
  - *Network I/O*: Transferring data between different machines, which can be a bottleneck.
  - *Deserialization and memory allocation*: Converting data back into a usable format and allocating memory for it.
- **Spark schedules general task graphs**
  - Spark automatically organizes tasks into a graph, optimizing the execution by pipelining functions, being aware of data locality, and minimizing unnecessary shuffles. This helps

in efficiently managing resources and reducing execution time.

13 / 16: Broadcast Variables

Broadcast Variables

- **Challenge**
  - Sending common variables to nodes with code can be costly
  - Involves serialization, network transfer, deserialization
  - Sending large, constant data repeatedly increases costs
- **Solution**
  - Cache read-only variables on each node, to avoid repeated transfers
- **Example**

```
# \textcolor{blue}{\texttt{var}} is large variable.
var = list(range(1, int(1e6)))
# Create a broadcast variable.
broadcast_var = sc.broadcast(var)
# Do not modify \textcolor{blue}{\texttt{var}}, but use \textcolor{bl
# of \textcolor{blue}{\texttt{var}}.
```

13 / 16

- **Broadcast Variables**
- **@Challenge@**
  - When working with distributed computing systems, like those used in big data processing, we often need to send common variables to multiple nodes. This can be quite costly because it involves several steps: *serialization* (converting the variable into a format that can be sent over the network), *network transfer* (actually sending the data), and *deserialization* (converting it back to its original format on the receiving end).
  - If the data is large and needs to be sent repeatedly, these costs can add up quickly, making the process inefficient.
- **@Solution@**
  - To address this issue, we can use broadcast variables. These allow us to cache read-only variables on each node in the cluster. By doing this, we avoid the need to send the same data multiple times, which reduces the overhead and improves performance.
- **@Example@**
  - In the provided Python code snippet, we have a large variable `var` that contains a list of numbers. Instead of sending `var` to each node every time it's needed, we create a broadcast variable using `sc.broadcast(var)`.
  - This broadcast variable, `broadcast_var`, is then used across the nodes. It's important to note that `var` should not be modified after broadcasting. Instead, we use

`broadcast_var.value` to access the data, ensuring consistency and efficiency.

14 / 16: Spark: Accumulators

Spark: Accumulators

- **Accumulator** is a shared variable used for aggregating values across tasks
  - Updated using associative and commutative operations (e.g., sum, max)
  - Efficient in parallel systems like MapReduce
- **Usage Example**
  - Accumulator initialized on the driver
  - Updated inside transformations (e.g., `foreach`) across workers
  - Value is collected back to the driver
  - Accumulator only guaranteed to update once per action
- **Example**

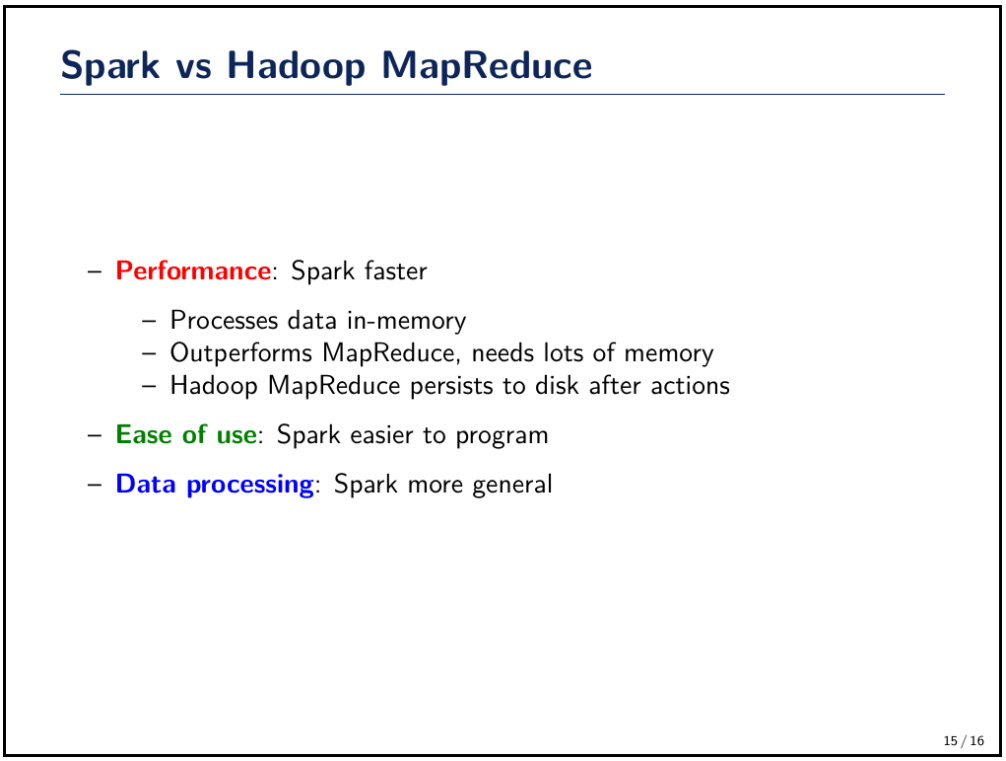
```
>>> accum = sc.accumulator(0)
>>> sc.parallelize([1, 2, 3, 4]).foreach(lambda x: accum.add(x))
>>> accum.value
10
```

14 / 16

- **Spark: Accumulators**
  - *Accumulators* are special variables in Spark that help in aggregating values across different tasks. Think of them as a way to keep a running total or summary of data as it's processed in parallel.
  - They are updated using operations that are both associative (grouping doesn't change the result) and commutative (order doesn't change the result), like addition or finding the maximum value. This makes them very efficient in distributed systems like Spark, which is built on the MapReduce model.
- **Usage Example**
  - An *accumulator* is first set up on the driver, which is the main program that controls the Spark application.
  - It is then updated within transformations, such as `foreach`, which are operations applied to each element of an RDD (Resilient Distributed Dataset) across different worker nodes.
  - After processing, the accumulated value is sent back to the driver, where it can be accessed.
  - It's important to note that an accumulator is only guaranteed to update once per action, meaning its value is reliable only after an action like `collect` or `count` is called.
- **Example**
  - In the provided Python code snippet, an accumulator is initialized with a starting value of 0.
  - The `parallelize` function creates an RDD from a list of numbers, and `foreach` is used to add each number to the accumulator.

- After processing, the value of the accumulator is 10, which is the sum of the numbers in the list. This demonstrates how accumulators can be used to efficiently sum values across distributed tasks.

15 / 16: Spark vs Hadoop MapReduce



- **Performance**: Spark faster
  - *Processes data in-memory*: Spark is designed to keep data in memory between operations, which significantly speeds up processing times. This is because accessing data from memory is much faster than reading from disk.
  - *Outperforms MapReduce, needs lots of memory*: While Spark is generally faster than Hadoop MapReduce, it requires a substantial amount of memory to achieve this performance. This can be a limitation if resources are constrained.
  - *Hadoop MapReduce persists to disk after actions*: In contrast, Hadoop MapReduce writes intermediate results to disk after each map or reduce action. This makes it slower, especially for iterative tasks, but it can handle larger datasets that don't fit into memory.
- **Ease of use**: Spark easier to program
  - Spark provides a more user-friendly API, which makes it easier for developers to write and understand code. This is particularly beneficial for those who are new to big data processing.
- **Data processing**: Spark more general
  - Spark is not limited to just batch processing like MapReduce. It supports a wide range of data processing tasks, including real-time stream processing, interactive queries, and machine learning, making it a more versatile tool for various data processing needs.

16 / 16: Gray Sort Competition

Gray Sort Competition

	@Hadoop MR Record@	@Spark Record (2014)@
Data Size	102.5 TB	100 TB
Elapsed Time	72 mins	23 mins
# Nodes	2100	206
# Cores	50400 physical	6592 virtualized
Cluster disk throughput	3150 GB/s	618 GB/s
Network	dedicated data center, 10Gbps	virtualized (EC2) 10Gbps network
Sort rate	1.42 TB/min	4.27 TB/min
Sort rate/node	0.67 GB/min	20.7 GB/min

- Sort benchmark, Daytona Gray: sort 100 TB of data (1 trillion records)
  - Spark-based System 3x faster with 1/10 the number of nodes
  - Speed up is ~30x
- Ref

16 / 16

- **Gray Sort Competition**: This slide presents a comparison between two systems, Hadoop MapReduce (MR) and Spark, in a sorting benchmark known as the Daytona Gray Sort. The goal of this benchmark is to sort a massive amount of data, specifically 100 terabytes (TB), which equates to about 1 trillion records.
- **Data Size**: Hadoop MR sorted 102.5 TB, while Spark sorted 100 TB. Both systems handled a similar scale of data, but Spark achieved this with slightly less data volume.
- **Elapsed Time**: Hadoop MR took 72 minutes to complete the task, whereas Spark finished in just 23 minutes. This highlights Spark's efficiency and speed in processing large datasets.
- **Number of Nodes**: Hadoop MR used 2100 nodes, while Spark only required 206 nodes. This indicates that Spark is more resource-efficient, achieving the task with significantly fewer nodes.
- **Number of Cores**: Hadoop MR utilized 50,400 physical cores, compared to Spark's 6,592 virtualized cores. Spark's use of virtualized cores on fewer nodes demonstrates its ability to optimize computational resources.
- **Cluster Disk Throughput**: Hadoop MR had a throughput of 3150 GB/s, while Spark's was 618 GB/s. Despite the lower throughput, Spark's overall performance was superior due to its efficient processing capabilities.
- **Network**: Hadoop MR operated in a dedicated data center with a 10Gbps network, whereas Spark ran on a virtualized EC2 environment with the same network speed. This shows Spark's adaptability to different environments.
- **Sort Rate**: Hadoop MR achieved a sort rate of 1.42 TB/min, while Spark reached 4.27

TB/min. Spark's sort rate was significantly higher, demonstrating its faster processing speed.

- **Sort Rate/Node**: Hadoop MR had a sort rate of 0.67 GB/min per node, whereas Spark achieved 20.7 GB/min per node. This highlights Spark's efficiency in utilizing each node's capacity.
- **Conclusion**: The Spark-based system was approximately three times faster than Hadoop MR and used only one-tenth of the nodes, resulting in an overall speedup of about 30 times. This showcases Spark's superior performance in large-scale data processing tasks.
- **Reference**: For more details, you can refer to the blog post from Databricks here.